Metro Ticket Booking Digitalization

Phase 5: Project Development & Configuration

1. Introduction to the Build

The Development Phase translates the design requirements into active ServiceNow configurations. This stage involves setting up the core data dictionary, scripting client interactions, and constructing the automated background pipelines in the DEV environment.

2. Core Configuration Components

Custom Tables & ACLs

Created `u_metro_station` and `u_ticket_booking`. Access Control Lists (ACLs) were configured to ensure commuters can only read station data and insert booking records, restricting update/delete rights to the admin role.

Catalog Item UI

Constructed a Record Producer in the Service Portal. Added variables for *Origin*, *Destination* (Reference fields), and *Passengers*. Applied UI Policies to hide the submit button until all required fields are validated.

Client Scripts & GlideAjax

Implemented an onChange Client Script on the destination field to prevent users from selecting the same station as their origin, alerting them immediately via `g_form.showFieldMsg()`.

Script Include (Backend)

Developed a client-callable Script Include named `FareCalculatorAjax`. This queries the matrix and returns the expected cost back to the portal form in real time.

3. GlideAjax Code Snippet (Example)

```
var FareCalculatorAjax = Class.create(); FareCalculatorAjax.prototype = Object.extendsObject(AbstractAjaxProcessor, { getFare: function() { var origin = this.getParameter('sysparm_origin'); var dest = this.getParameter('sysparm_dest'); // Query u_fare_matrix to calculate base cost return calculatedCost; } });
```

4. Flow Designer Orchestration

The backend logic was successfully moved away from legacy Business Rules into a streamlined Flow Designer flow. This visual representation ensures ease of maintenance and immediate debugging capabilities.

Step	Action Type	Configuration Details
1	Trigger	Fires when a record is *Created* on `u_ticket_booking`.
2	REST Step (Int. Hub)	Executes a POST method to a third-party QR API, parsing the `sys_id` and commuter details as the JSON payload.
3	Update Record	Takes the returned URL from the API response and updates the `u_qr_image_link` field on the created ticket record.

5. Notifications Strategy

- **Email Alert:** Configured a System Notification triggering on the update of the `u_qr_image_link` field. It utilizes a Mail Script to embed the generated QR image directly into the HTML body of the email.
- **SMS Alert:** Implemented a Twilio spoke integration to fire a brief SMS to the user's registered phone number containing a hyperlink to their digital ticket.

6. Quality Assurance & Testing

Post-development, the system underwent rigorous testing in the TEST instance. Developers executed unit tests using the Automated Test Framework (ATF) to simulate 100 concurrent ticket bookings, verifying that the Flow Designer queue processed all REST calls without instance degradation. User Acceptance Testing (UAT) validated that the mobile portal correctly scaled on smaller screens without horizontal scrolling.